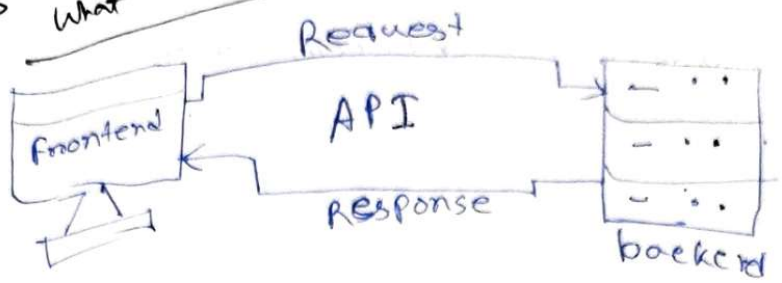
FAST API

What is Fast API?

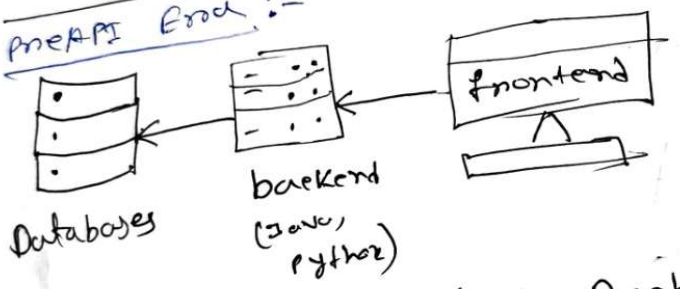
=> What is API :-

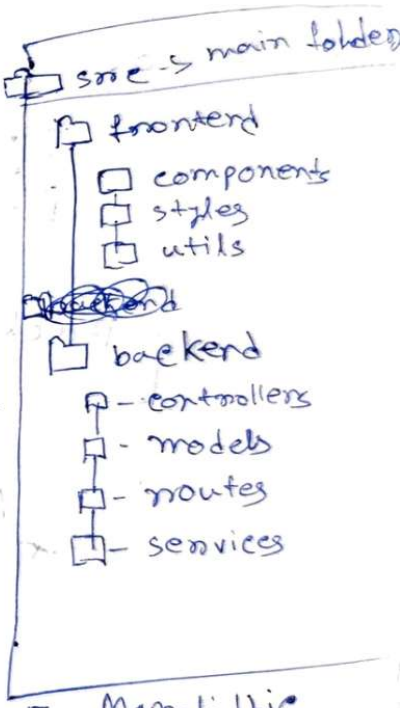


APIs are mechanism that enable two software components - such as the ~~fr~~ frontend and backend of an application - to communicate with each other using a define set of rules, protocols (HTTP), and data formats.

Need for APIs

pre API Era :-



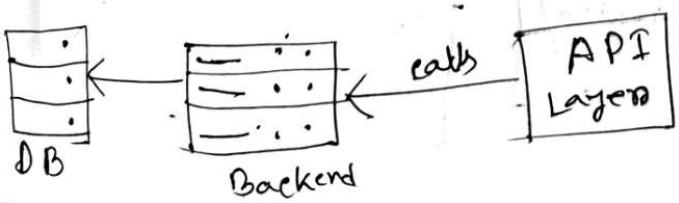


Monolithic Architecture

Draw back of Monolithic Architecture :-

In pre API era whole APP in one folde tightly, thats why if any error in one particular thing, it can affect whole app system.

After API :-



we do main two different things here :

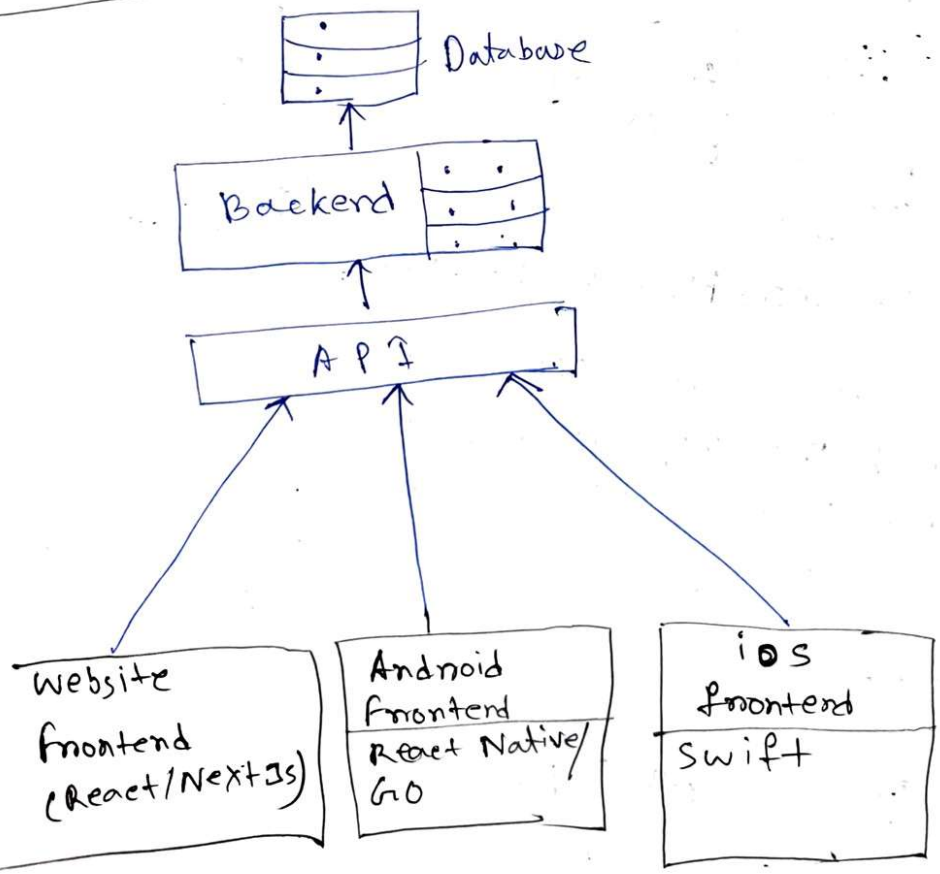
→ Remove frontend

→ we cannot give access our backend directly here we give Access only our API.

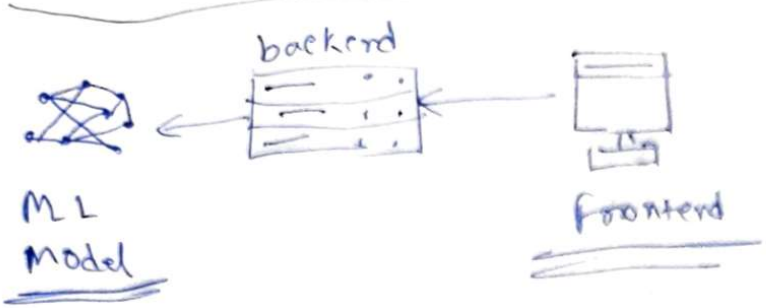
HTTP:- In here we use HTTP protocol

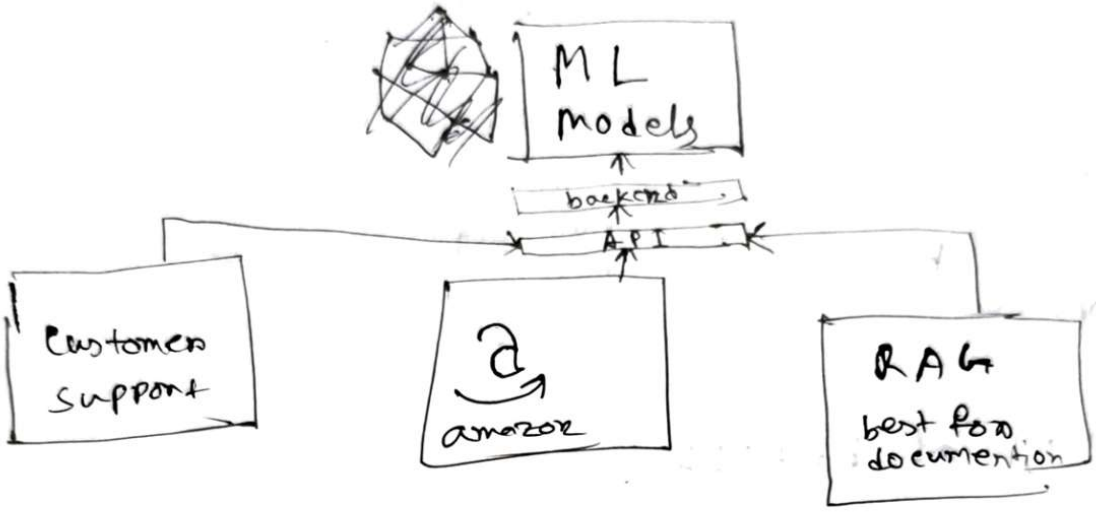
JSON:- JSON is a Data format which API gives to our application form of Data. It is a universal format which under-stand by any language

Benifits of using API :-



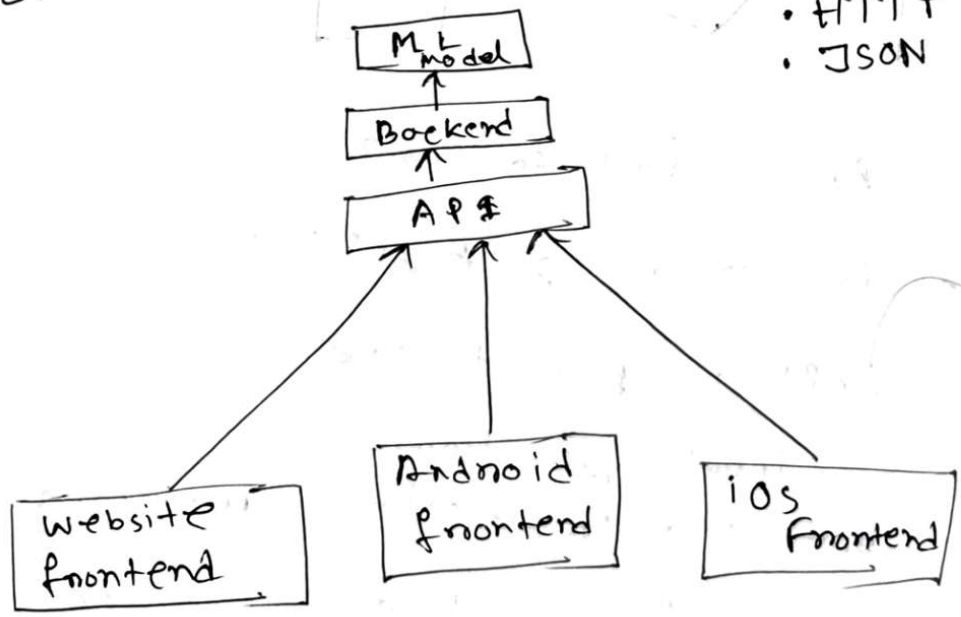
API- ML Perspective :-





- Everything is same as software, here is ~~like~~ we have ML models rather than databases. ~~this~~ this is the only part of ~~the~~ difference other than everything is same.

- HTTP protocol
- JSON Data format



Video-2 (CampusX)

FASTAPI:

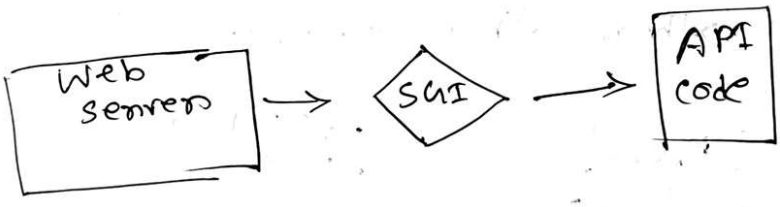
Fastabi is a modern, high-performance web framework for building APIs with python

- Its created by mainly two library.

• Starlette	Pydantic
Starlette manages how your API recives requests and sends back responses.	Pydantic is used to check if the data coming into your API is correct and in the right format

Philosophy of FastAPI:

- fast to Run
- fast to code



- Why FastAPI is fast to run?

=> beause it use mainly Async nature

- why FastAPI is Fast to code?

=> • Automatic input validation.
   • Auto-Generated interactive Documentation

- seamless integration with Modern Ecosystem (ML/DL libraries, OAuth, JWT, SQL Alchemy, Docker, kubernetes etc).

Terminal :-

01 - Step :-

create anew virtual enviornment

Terminal :-

```
python -m venv myenv
```

~~next acti~~

02 - Step :-

next activat fresh virtual enviornment :-

Terminal :-

```
myenv\Scripts\activate
```

03 - Step :-

If you want to check you are in or not :-

Terminal :-

```
where.exe python
```

04 - Step :-

for running the code :-

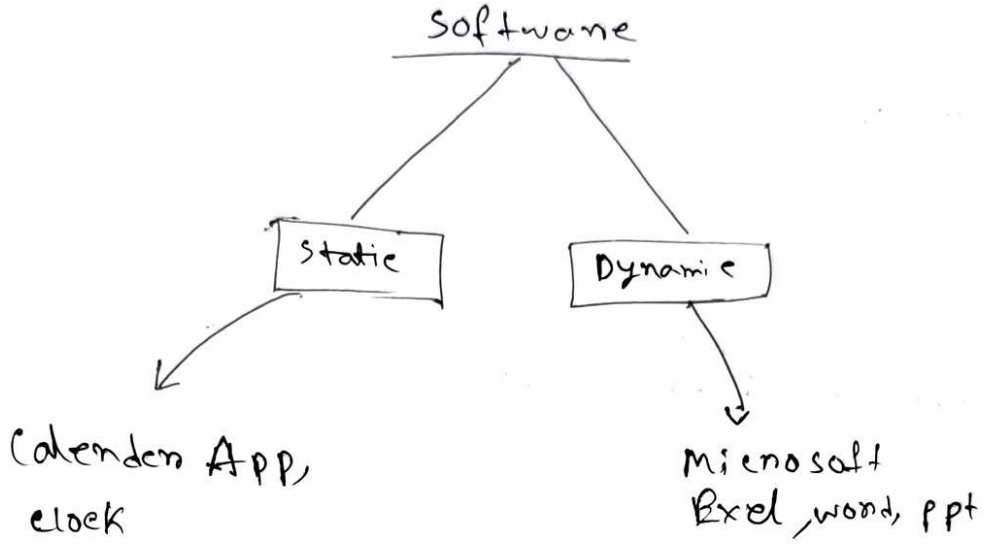
Terminal :-

uvicorn folldermame: app Name space space -- reload

```
uvicorn main:app -- reload
```

Video-3 (campus X)

HTTP Methods :-



Dynamic software :- CRUD

create, Retrive, update, Delete.

total 4 type of interaction.

HTTP 4 major methods are :-

Get | POST | PUT | DELETE

Video - 4 (campusx)

Path Params

Path parameters are dynamic segments of a URL path use to identify a specific resource.

we use mainly path-parameter in 3 cases:

- retrive
- update
- delete

Path function :-

The path () function in FASTAPI is used to provide metadata, validation rules, and documentation hints for path parameters in your API end points.

Title, Description, Example, ge, gt, le, lt, Main_length, Max_length, regex.

HTTP status code :-

2XX :- success, The request was successfully recived and processed.

3XX : Redirection, further action needs to be taken (e.g, redirect)

4XX : client Error, Something is wrong with the request from the client.

5XX → server Error, something went wrong on the server side.

Some famous status code :

- **200 OK** : Standard success , A get or post succeed.
- **201 Created** : Resource created , After a post that creates something.
- **204 No Content** : Success, but no data returned , After a DELETE request
- **400 Bad Request** : Malformed or invalid authentication , Missing field, wrong data type.
- **401 Unauthorized** : No/invalid authentication , Login required.
- **403 Forbidden** : Authenticated, but no permission , Logged in but not allowed.
- **404 Not Found** : Resource doesn't exist , Patient ID not in DB.
- **500 Internal Server Error** : Generic failure , Something broke on the server
- **502 Bad Gateway** : Gateway (like Nginx) failed to reach backend
- **503 Service unavailable** : Server is down or overloaded.

HTTP Exception (fune) :-

HTTPException is a special built-in exception in FASTAPI used to return custom HTTP error responses , when something goes wrong in your API.

Instead of returning a normal JSON or crashing the server, you can gracefully raise an error with:

- a proper HTTP status code (like 404, 403, 400)
- a custom error message.
- (optional) extra headers.

Query Parameters :-

query parameters are optional key-value pairs appended to the end of a URL, used to pass additional data to the server in a HTTP request. They are typically employed for operations like filtering, sorting, searching, and pagination, without altering the endpoint path itself.

/patients?city=Delhi&short_by=age

URL Key Value Key Value

- The ? marks the start of query parameters.
- Each parameter is a key-value pair : key=value
- multiple parameters are separated by &

in this case:

- city = Delhi is a query parameter for filtering
- sort_by = age is a query parameter for sorting

Query() is a utility function provided by FastAPI to declare, validate and document query parameters in your API endpoints.

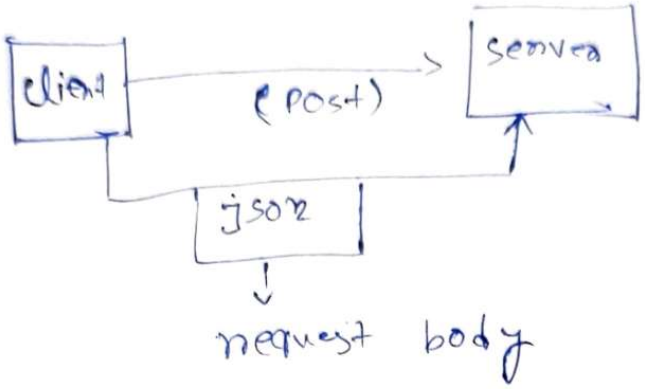
- you can also add using query() function :-

default, title, ~~descr~~ description, example / examples

~~min leng~~ min_length, max_length, ge, gt, le, lt

regex

Video-6 POST Request in FASTAPI



A request body is the portion of HTTP request that contains data sent by the client to the server. It is typically used in HTTP methods such as post, or PUT to transmit structure data (eg., JSON, XML, form-data) for the purpose of creating or updating resources on the server. The server parses the request body to extract the necessary information and perform the intended operation.

PUT is for Update

for Post we use how data add our Json file :-

```
Json.dump(data, f)
```

Video-7 / Update / DELETE

(/edit) HTTP.PUT

Patient id

request body

{city: "new value"}

{weight: "new value"}

Step-1:-

we will build new pydantic model

Step-2:

new data add to the existing olden pydantic model.